گزارش نهایی پروژه عملی رمزنگاری

پیاده‌سازی رمز جریانی مبتنی بر LFSR و پخش رمزگذاری‌شده ویدئو

نام دانشجو: محمد متین سلیمی
شماره دانشجویی: ۴۰۲۲۴۳۰۷۴
نام درس: رمزنگاری
عنوان پروژه: پخش ویدئو با رمز جریانی مبتنی بر LFSR

۱. مقدمه و هدف پروژه

هدف این پروژه آشنایی عملی با رمزهای جریانی، نحوه تولید دنباله کلید با استفاده از ثبات بازخورد خطی (LFSR)، بررسی ساختارهای غیرخطی و استفاده از این روش‌ها در یک سامانه پخش ویدئوی رمزگذاری‌شده است.

در رمزهای جریانی، به‌جای رمزگذاری کل پیام به‌صورت یکجا، یک دنباله شبه‌تصادفی از بیت‌ها تولید می‌شود که به آن دنباله کلید گفته می‌شود. سپس این دنباله با داده اصلی ترکیب می‌شود تا متن رمز تولید شود.

در این پروژه، سرور قطعه‌های مختلف فایل ویدئویی را رمزگذاری کرده و در اختیار مرورگر قرار می‌دهد. مرورگر نیز با استفاده از یک سرویس پس‌زمینه از نوع Service Worker، قطعه‌های دریافتی را هنگام نیاز رمزگشایی می‌کند. به این ترتیب، پخش ویدئو می‌تواند پیش از دریافت کامل فایل آغاز شود و دریافت و رمزگشایی قطعه‌های بعدی هم‌زمان با پخش ادامه پیدا کند.

اهداف اصلی پروژه عبارت‌اند از:

- پیاده‌سازی یک مولد دنباله کلید مبتنی بر LFSR شانزده‌بیتی.
- پشتیبانی از تولید دنباله کلید از یک Offset مشخص.
- پیاده‌سازی عملیات رمزگذاری و رمزگشایی با استفاده از XOR.
- پیاده‌سازی دو ساختار NLFSR و Geffe برای مقایسه مولدهای خطی و غیرخطی.
- یکسان‌سازی رفتار پیاده‌سازی پایتون و جاوااسکریپت.
- بررسی صحت عملکرد با استفاده از آزمون‌های خودکار.
- ارزیابی سرعت تولید دنباله و بررسی ویژگی‌های آماری خروجی با آزمون‌های NIST.
- بررسی عملکرد پخش ویدئو به‌صورت قطعه‌ای در مرورگر.

۲. معماری کلی سامانه

سامانه از دو بخش اصلی تشکیل شده است:

۲-۱. بخش سرور

سرور فایل‌های ویدئویی موجود در پوشه ویدئوها را شناسایی کرده و فهرست آن‌ها را در اختیار رابط کاربری قرار می‌دهد. پس از انتخاب یک ویدئو، مرورگر درخواست دریافت بازه مشخصی از فایل را ارسال می‌کند.

سرور پس از اعتبارسنجی درخواست، بازه موردنظر را از فایل اصلی می‌خواند، با استفاده از دنباله کلید مبتنی بر LFSR رمزگذاری می‌کند و نتیجه را برای مرورگر ارسال می‌کند.

۲-۲. بخش مرورگر

در سمت مرورگر، Service Worker مسئول دریافت و پردازش درخواست‌های مربوط به ویدئو است. این سرویس قطعه‌های رمزگذاری‌شده را دریافت کرده و با استفاده از همان Seed و Offset، آن‌ها را رمزگشایی می‌کند.

پس از رمزگشایی، داده اصلی در اختیار فرایند پخش ویدئو قرار می‌گیرد.

۲-۳. مزیت معماری قطعه‌ای

در روش دریافت کامل فایل، کاربر باید تا پایان دانلود ویدئو منتظر بماند. اما در این پروژه، داده‌ها به قطعه‌های کوچک‌تر تقسیم شده‌اند و هر قطعه به‌صورت مستقل دریافت و رمزگشایی می‌شود.

مزایای این روش عبارت‌اند از:

- آغاز پخش پیش از دریافت کامل ویدئو.
- کاهش نیاز به نگهداری کل فایل در حافظه مرورگر.
- امکان دریافت بخش‌های موردنیاز هنگام جابه‌جایی در ویدئو.
- امکان پردازش قطعه‌ها به‌صورت مستقل.
- افزایش انعطاف‌پذیری سامانه در مدیریت درخواست‌های ویدئویی.

۳. مبانی نظری و الگوریتم LFSR

۳-۱. تعریف LFSR

ثبات بازخورد خطی یا Linear Feedback Shift Register یک ساختار دیجیتال است که از چند بیت حافظه تشکیل شده و در هر مرحله، با انجام عملیات شیفت و محاسبه یک بیت بازخورد، حالت جدیدی تولید می‌کند.

در این پروژه از یک LFSR شانزده‌بیتی استفاده شده است. مقدار اولیه ثبات با عنوان Seed مشخص می‌شود و با هر مرحله اجرای الگوریتم، یک بیت خروجی تولید می‌شود.

ویژگی مهم LFSR سادگی و سرعت بالای اجرای آن است. با این حال، به دلیل خطی‌بودن رابطه بازخورد، خروجی آن از نظر رمزنگاری امن محسوب نمی‌شود.

۳-۲. چندجمله‌ای بازخورد

چندجمله‌ای مورد استفاده در پروژه به شکل زیر است:

```
P(x) = x^16 + x^14 + x^13 + x^11 + 1
```

با توجه به قرارداد شیفت به راست و شماره‌گذاری بیت‌ها از صفر، موقعیت‌های مورد استفاده در رابطه بازخورد عبارت‌اند از:

```
5 ,3 ,2 ,0
```

در هر مرحله، بیت‌های متناظر با این موقعیت‌ها با یکدیگر XOR می‌شوند تا بیت بازخورد جدید محاسبه شود.

۳-۳. مراحل تولید هر بیت

فرایند تولید هر بیت به ترتیب زیر انجام می‌شود:

۱. کم‌ارزش‌ترین بیت ثبات به‌عنوان بیت خروجی انتخاب می‌شود.

۲. بیت بازخورد از XOR بیت‌های انتخاب‌شده محاسبه می‌شود.

۳. ثبات یک بیت به سمت راست شیفت داده می‌شود.

۴. بیت بازخورد در جایگاه شانزدهم (بیت ۱۵ با شماره‌گذاری صفرمبنا) قرار می‌گیرد.

رابطه محاسباتی بازخورد:

```
feedback = bit0 XOR bit2 XOR bit3 XOR bit5
```

رابطه به‌روزرسانی حالت:

```
next_state = (state >> 1) | (feedback << 15)
```

بیت خروجی نیز برابر است با:

```
output_bit = state & 1
```

۴-۳. بسته‌بندی بیت‌ها در بایت

از آنجا که داده‌های فایل به‌صورت بایت پردازش می‌شوند، لازم است هر هشت بیت تولیدشده در یک بایت قرار بگیرند.

در این پروژه، اولین بیت تولیدشده در پرارزش‌ترین موقعیت بایت قرار می‌گیرد. بنابراین بیت‌های خروجی به ترتیب در موقعیت‌های ۷ تا صفر قرار داده می‌شوند.

این قرارداد در هر دو پیاده‌سازی پایتون و جاوااسکریپت یکسان رعایت شده است.

۴. پیاده‌سازی توابع اصلی

در این بخش هشت تابع موردنیاز پروژه پیاده‌سازی شده‌اند.

۴-۱. تابع تولید یک بیت در پایتون

نام تابع:

```
next_bit(state)_
```

این تابع یک حالت شانزده‌بیتی را دریافت کرده و دو مقدار شامل بیت خروجی و حالت بعدی ثبات را برمی‌گرداند.

مراحل اجرای آن:

- بررسی معتبر بودن مقدار ورودی.
- استخراج کم‌ارزش‌ترین بیت به‌عنوان خروجی.
- محاسبه بازخورد با استفاده از XOR بیت‌های مشخص‌شده.
- شیفت ثبات به سمت راست.
- قرار دادن بیت بازخورد در بیت پرارزش ثبات.

۴-۲. تابع تولید دنباله کلید

نام تابع:

```
keystream(length, seed, offset)
```

این تابع مسئول تولید تعداد مشخصی بایت از دنباله کلید است.

پارامترهای آن عبارت‌اند از:

- `length` : تعداد بایت‌های موردنیاز.
- `seed` : مقدار اولیه ثبات.
- `offset` : محل شروع تولید دنباله بر حسب بایت.

برای پشتیبانی از Offset، ابتدا الگوریتم به‌اندازه هشت برابر Offset مرحله اجرا می‌شود تا ثبات به موقعیت موردنظر برسد. سپس بایت‌های موردنیاز تولید می‌شوند.

این قابلیت برای رمزگشایی قطعه‌های مستقل فایل ویدئویی اهمیت زیادی دارد؛ زیرا هر قطعه باید دقیقاً با همان بخش از دنباله کلید که هنگام رمزگذاری استفاده شده است، پردازش شود.

۴-۳. تابع رمزگذاری و رمزگشایی داده‌ها

نام تابع:

```
xor_bytes(data, seed, offset)
```

این تابع داده ورودی را با دنباله کلید متناظر XOR می‌کند.

رابطه رمزگذاری:

```
C = P XOR K
```

رابطه رمزگشایی:

```
P = C XOR K
```

در این روابط:

- `P` : متن اصلی.
- `C` : متن رمز.
- `K` : دنباله کلید.

از آنجا که XOR یک عملگر متقارن است، اجرای دوباره همین عملیات با همان دنباله کلید، داده اصلی را بازیابی می‌کند.

۴-۴. پیاده‌سازی در جاوااسکریپت

برای آنکه مرورگر بتواند داده‌های دریافتی را رمزگشایی کند، سه تابع متناظر با پیاده‌سازی پایتون در فایل جاوااسکریپت نیز پیاده‌سازی شدند:

```
nextBit(state)
keystream(length, seed, offset)
xorBytes(data, seed, offset)
```

در تابع `xorBytes` از ساختار `Uint8Array` برای پردازش داده‌های باینری استفاده شده است.

نکته مهم در این قسمت، یکسان‌بودن ترتیب تولید بیت‌ها، شیفت ثبات، محاسبه Offset و بسته‌بندی بایت‌ها در هر دو زبان است.

در نتیجه، داده‌ای که توسط سرور با پایتون رمزگذاری شده است، می‌تواند در مرورگر با جاوااسکریپت به‌درستی رمزگشایی شود.

۵. پیاده‌سازی مولدهای غیرخطی

۵-۱. مولد NLFSR

در ساختار NLFSR یا Nonlinear Feedback Shift Register، علاوه بر بیت‌های بازخورد پایه، یک جمله غیرخطی نیز به رابطه بازخورد اضافه می‌شود.

در این پروژه جمله غیرخطی به شکل زیر تعریف شده است:

```
nonlinear_term = bit1 AND bit7
```

رابطه بازخورد جدید:

```
feedback = bit0 XOR bit2 XOR bit3 XOR bit5 XOR (bit1 AND bit7)
```

وجود عملگر AND باعث می‌شود رابطه بازخورد دیگر صرفاً یک رابطه خطی نباشد.

هدف از این تغییر، بررسی تأثیر اضافه‌شدن یک جمله غیرخطی بر ویژگی‌های آماری و سرعت تولید دنباله است.

با این حال، غیرخطی‌شدن رابطه بازخورد به‌تنهایی تضمین‌کننده امنیت رمزنگاری نیست. اندازه حالت، دوره مولد و مقاومت در برابر حملات همچنان اهمیت زیادی دارند.

۵-۲. مولد Geffe

مولد Geffe یکی از ساختارهای ترکیبی است که خروجی سه LFSR را با استفاده از یک تابع غیرخطی ترکیب می‌کند.

در این پروژه سه مولد مستقل اجرا شده و خروجی آن‌ها با تابع Geffe ترکیب شده است.

رابطه تابع ترکیب:

```
output = (x AND y) XOR ((NOT x) AND z)
```

در این رابطه:

- x خروجی مولد اول است.
- y خروجی مولد دوم است.
- z خروجی مولد سوم است.

در صورتی که مقدار x برابر یک باشد، خروجی مولد دوم انتخاب می‌شود و در غیر این صورت خروجی مولد سوم انتخاب خواهد شد.

این ساختار نسبت به یک LFSR ساده پیچیده‌تر است، اما همچنان ممکن است در برابر حملات همبستگی آسیب‌پذیر باشد.

۶. آزمون صحت عملکرد و تست‌های خودکار

پس از تکمیل توابع، مجموعه آزمون‌های خودکار پروژه اجرا شد.

دستور اجرا:

```
python -m unittest discover -s tests -v
```

نتیجه اجرای آزمون‌ها:

```
Ran 20 tests in 1.906s

OK
```

در مجموع ۲۰ آزمون با موفقیت اجرا شدند.

این آزمون‌ها بخش‌های مختلف پیاده‌سازی را بررسی می‌کنند، از جمله:

- صحت تولید دنباله کلید.

- صحت بردار تست مشخص‌شده در پروژه اولیه.
- تقارن عملیات XOR.
- صحت محاسبه Offset.
- اعتبارسنجی ورودی‌ها و مدیریت خطاها.
- عملکرد مولدهای NLFSR و Geffe.
- یکسان‌بودن خروجی پایتون و جاوااسکریپت.
- عملکرد بخش‌های مرتبط با سرور و Service Worker.
- صحت عملکرد رابط کاربری.

نتیجه موفقیت‌آمیز این آزمون‌ها نشان می‌دهد توابع پیاده‌سازی‌شده با تست‌های موجود در پروژه سازگار هستند.

۷. ارزیابی سرعت و مقایسه مولدها

برای مقایسه عملکرد مولدها از ابزار ارزیابی پروژه استفاده شد.

در این آزمایش، سه مولد LFSR، NLFSR و Geffe با اندازه داده یکسان مورد بررسی قرار گرفتند.

۷-۱. مقایسه سرعت تولید دنباله

نتایج به‌دست‌آمده:

مولد	سرعت تولید (Mbps)
LFSR	1.466
NLFSR	1.600
Geffe	0.546

بر اساس نتایج این اجرا، مولد NLFSR با سرعت حدود ۱,۶۰۰ مگابیت بر ثانیه سریع‌ترین مولد بوده است.

مولد LFSR با سرعت حدود ۱,۴۶۶ مگابیت بر ثانیه در جایگاه دوم قرار گرفته است.

مولد Geffe نیز با سرعت حدود ۰,۵۴۶ مگابیت بر ثانیه کندترین روش بوده است.

کاهش سرعت Geffe را می‌توان به اجرای سه LFSR و ترکیب خروجی‌های آن‌ها نسبت داد.

این نتایج مربوط به محیط اجرای آزمایش هستند و ممکن است با تغییر سخت‌افزار، اندازه داده یا شرایط اجرا تغییر کنند.

۲-۷. بررسی عملکرد رمزگذاری برای پخش ویدئو

در بنچمارک اصلی، مشخصات زیر اندازه‌گیری شد:

شاخص	مقدار
حجم داده	۸۱۹۲ بایت
تعداد قطعه‌ها	۲
میانگین زمان پردازش هر قطعه	۳۱٫۸۸ میلی‌ثانیه
زمان کل اندازه‌گیری	۰٫۰۶۳۷۶۵ ثانیه
نرخ رمزگذاری	۱٫۰۲۸ Mbps
نرخ فرضی ویدئو	۲ Mbps
نسبت ظرفیت به نرخ ویدئو	۰٫۵۱۴

برای پخش بی‌درنگ، نرخ پردازش رمزگذاری و رمزگشایی باید در شرایط واقعی حداقل با نرخ مصرف داده ویدئو قابل مقایسه باشد. نسبت ظرفیت به نرخ ویدئو در این بنچمارک کمتر از یک است؛ بنابراین بر اساس همین اندازه‌گیری، ظرفیت پردازش رمزگذاری به نرخ فرضی ویدئو نمی‌رسد.

با وجود این، در اجرای عملی پروژه ویدئو با موفقیت پخش شد. این موضوع نشان می‌دهد که معیار بنچمارک به‌تنهایی برای پیش‌بینی رفتار کل سامانه کافی نیست؛ زیرا پخش واقعی به عواملی مانند اندازه قطعه‌ها، بافر مرورگر، زمان‌بندی درخواست‌ها و شرایط پردازش نیز وابسته است.

۸. بررسی آزمون‌های آماری NIST SP 800-22

برای بررسی ویژگی‌های آماری دنباله‌های تولیدشده، از کتابخانه `nistrng` و سه آزمون زیر استفاده شد:

۸-۱. آزمون Monobit

این آزمون نسبت تعداد بیت‌های صفر و یک را بررسی می‌کند. هدف آن تشخیص عدم تعادل آشکار میان صفرها و یک‌ها است.

۸-۲. آزمون Runs

این آزمون تعداد دنباله‌های متوالی از بیت‌های یکسان را بررسی می‌کند. تعداد این دنباله‌ها باید با انتظار آماری متناسب باشد.

۸-۳. آزمون DFT

این آزمون با استفاده از تبدیل فوریه گسسته، وجود الگوهای تناوبی و ساختارهای غیرتصادفی در دنباله را بررسی می‌کند.

۸-۴. نتایج آزمون‌ها

مولد	Monobit	Runs	DFT
LFSR	قبول	قبول	رد
NLFSR	قبول	قبول	رد
Geffe	قبول	قبول	رد

نتایج عددی گزارش‌شده توسط ابزار:

مولد	Monobit Score	Runs Score	DFT Score
LFSR	0.9937666	0.9937664	0
NLFSR	0.7134792	0.1111094	تقریباً صفر
Geffe	0.9937666	0.9937664	بسیار کوچک

در این اجرا، هر سه مولد در آزمون‌های Monobit و Runs نتیجه قبولی دریافت کردند، اما هر سه در آزمون DFT رد شدند.

مقادیر Score بیانگر مقادیر آماری گزارش‌شده توسط ابزار هستند و نباید به‌عنوان درصد امنیت یا احتمال امن‌بودن مولد تفسیر شوند.

همچنین این آزمایش تنها سه آزمون منتخب از مجموعه NIST را اجرا کرده است و اجرای کامل مجموعه SP 800-22 محسوب نمی‌شود.

۸-۵. چرا قبولی در آزمون‌های آماری امنیت را ثابت نمی‌کند؟

آزمون‌های آماری تنها برخی ویژگی‌های ظاهری و توزیعی یک دنباله را بررسی می‌کنند.

ممکن است یک مولد در چندین آزمون آماری قبول شود، اما ساختار داخلی آن همچنان قابل تحلیل و پیش‌بینی باشد.

برای مثال، LFSR به دلیل خطی‌بودن رابطه بازخورد، در برابر روش‌های تحلیل ساختاری آسیب‌پذیر است. بنابراین قبولی در آزمون‌هایی مانند Monobit و Runs به معنای مقاومت در برابر حملات رمزنگاری نیست.

امنیت یک مولد باید از نظر ساختار ریاضی، اندازه حالت، دوره، پیچیدگی خطی و مقاومت در برابر حملات نیز بررسی شود.

۹. محدودیت‌های امنیتی مولدها

۹-۱. محدودیت LFSR

LFSR ساده ساختار خطی دارد و با وجود سرعت مناسب، برای رمزنگاری واقعی کافی نیست.

اگر تعداد کافی از بیت‌های خروجی در اختیار مهاجم قرار بگیرد، ممکن است بتواند رابطه بازخورد را تحلیل کرده و بیت‌های بعدی را پیش‌بینی کند.

۹-۲. محدودیت NLFSR

اضافه‌کردن جمله غیرخطی می‌تواند ساختار مولد را پیچیده‌تر کند، اما اندازه حالت همچنان محدود است.

در این پروژه، حالت مولد تنها شانزده بیت است. بنابراین فضای حالت آن کوچک بوده و دوره و ویژگی‌های آماری آن به طراحی دقیق رابطه بازخورد وابسته است.

غیرخطی‌بودن به‌تنهایی برای تضمین امنیت کافی نیست.

۹-۳. محدودیت مولد Geffe

مولد Geffe خروجی سه LFSR را با یک تابع غیرخطی ترکیب می‌کند. با این حال، خروجی آن ممکن است با برخی از مولدهای داخلی همبستگی داشته باشد.

این ضعف می‌تواند در حملات همبستگی مورد استفاده قرار بگیرد و باعث شود مهاجم با تحلیل خروجی، اطلاعاتی درباره حالت داخلی مولدها به دست آورد.

بنابراین ترکیب چند LFSR نیز به‌تنهایی تضمین‌کننده امنیت نیست.

۹-۴. خطر استفاده مجدد از Seed

یکی از مشکلات مهم رمزهای جریانی، استفاده مجدد از یک دنباله کلید برای رمزگذاری دو پیام متفاوت است.

اگر دو پیام با یک Seed و Offset یکسان رمز شوند، دنباله کلید یکسانی تولید خواهد شد.

رابطه رمزگذاری پیام اول:

```
C1 = P1 XOR K
```

رابطه رمزگذاری پیام دوم:

```
C2 = P2 XOR K
```

با XOR کردن دو متن رمز:

```
C1 XOR C2 = P1 XOR P2
```

در نتیجه، دنباله کلید حذف شده و رابطه میان دو متن اصلی آشکار می‌شود.

این موضوع می‌تواند اطلاعات ارزشمندی درباره پیام‌ها در اختیار مهاجم قرار دهد.

به همین دلیل، استفاده مجدد از یک دنباله کلید در رمزهای جریانی خطرناک است.

۱۰. اجرای عملی و بررسی پخش ویدئو

پس از تکمیل توابع، سرور پروژه با دستور زیر اجرا شد:

```
python -m stream_server --port 8000
```

سپس رابط کاربری در نشانی زیر باز شد:

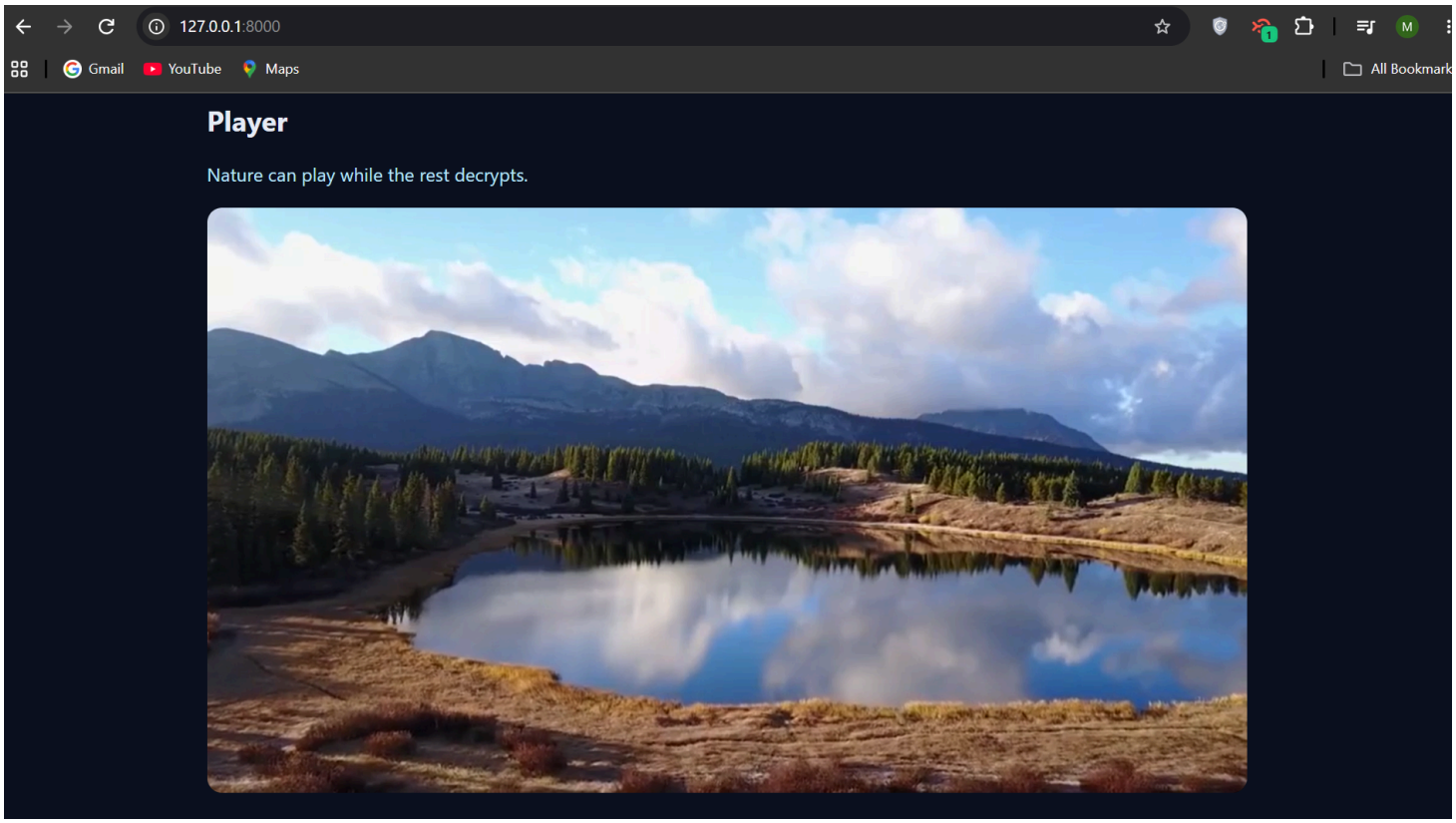
```
http://127.0.0.1:8000
```

یکی از ویدئوهای موجود در فهرست انتخاب شد و فرایند پخش مورد بررسی قرار گرفت.

نتیجه اجرای عملی:

- سرور با موفقیت راه‌اندازی شد.
- فهرست ویدئوها در رابط کاربری نمایش داده شد.
- ویدئوی انتخاب‌شده با موفقیت دریافت و پخش شد.
- رمزگشایی قطعه‌های دریافتی در مرورگر انجام شد.
- پخش ویدئو بدون نیاز به دریافت کامل فایل آغاز شد.
- دریافت و پردازش قطعه‌های بعدی در حین پخش ادامه یافت.

تصویر اجرای عملی پروژه



۱۱. بررسی اختلاف بردار تست در صورت تمرین و پروژه اولیه

در متن صورت تمرین، بردار تست زیر برای LFSR ذکر شده است:

```
Seed = 0xACE0
Expected output = 2e 44 35 87
```

اما در تست‌های پروژه اولیه، بردار زیر بررسی می‌شود:

```
Seed = 0xACE1
Expected output = 87 35 44 e2
```

این دو بردار با یکدیگر متفاوت هستند.

پیاده‌سازی انجام‌شده مطابق قرارداد صریح الگوریتم، یعنی خروجی LSB، شیفت به راست و قرارگرفتن نخستین بیت تولیدشده در پرارزش‌ترین موقعیت بایت، انجام شده است.

با توجه به اینکه تست‌های خودکار پروژه اولیه بردار دوم را بررسی می‌کنند، پیاده‌سازی با تست‌های موجود پروژه سازگار است.

این اختلاف میان متن صورت تمرین و تست‌های پروژه اولیه به‌عنوان یک ناسازگاری در مشخصات ثبت شده است و لازم است در صورت نیاز با مدرس پروژه بررسی شود.

۱۲. نتیجه‌گیری نهایی

در این پروژه، یک سامانه پخش ویدئوی رمزگذاری‌شده مبتنی بر رمز جریانی پیاده‌سازی و آزمایش شد.

در بخش اول، LFSR شانزده‌بیتی برای تولید دنباله کلید پیاده‌سازی شد. سپس با استفاده از عملیات XOR، توابع رمزگذاری و رمزگشایی داده‌ها توسعه داده شدند. برای پشتیبانی از پخش قطعه‌ای، قابلیت Offset نیز در تولید دنباله کلید در نظر گرفته شد.

در ادامه، دو ساختار NLFSR و Geffe برای بررسی تأثیر غیرخطی‌بودن و ترکیب چند مولد پیاده‌سازی شدند. نتایج ارزیابی نشان داد که در اجرای انجام‌شده، NLFSR سریع‌ترین و Geffe کندترین مولد بوده است.

آزمون‌های آماری NIST نیز نشان دادند که هر سه مولد در آزمون‌های Monobit و Runs قبول شدند، اما در آزمون DFT نتیجه قابل‌قبولی دریافت نکردند. این نتایج نشان می‌دهند که بررسی آماری برای شناخت ویژگی‌های دنباله مفید است، اما برای اثبات امنیت رمزنگاری کافی نیست.

در نهایت، اجرای عملی سامانه موفق بود و ویدئو بدون نیاز به دریافت کامل فایل، در مرورگر پخش شد. دریافت و رمزگشایی قطعه‌های بعدی نیز در حین پخش ادامه پیدا کرد.

این پروژه نشان داد که پیاده‌سازی یک رمز جریانی تنها شامل تولید بیت‌های شبه‌تصادفی نیست؛ بلکه هماهنگی دقیق میان تولید دنباله کلید، مدیریت Offset، پردازش داده‌های باینری، سرور و مرورگر نیز برای عملکرد صحیح سامانه ضروری است.

جمع‌بندی: اهداف آموزشی پروژه در زمینه پیاده‌سازی LFSR، بررسی مولدهای غیرخطی، رمزگذاری قطعه‌ای و ارزیابی آماری محقق شدند. با این حال، مولدهای پیاده‌سازی‌شده ساختارهای آموزشی هستند و نباید برای حفاظت از اطلاعات حساس در سامانه‌های واقعی استفاده شوند.

لینک گیت هاب پروژه : https://github.com/matinmms/secure_video_steam.git